Nestlé Distributed Order Management — Technical Report

WISER Global Quantum+AI Program 2026 — Nestlé Challenge

---

1. Business Importance

Nestlé fulfills customer orders from a network of plants, each with finite inventory, dock capacity, and throughput. When a default plant cannot fully serve an order, a planner must decide whether to redirect it to another plant, partially fulfill it, or reject it. Today this is largely rule-based and doesn't systematically weigh revenue against shipping cost across the whole network.

This is a combinatorial assignment problem: every order-to-plant pairing is a binary decision, constraints couple orders through shared plant inventory, and the objective balances competing terms. Problems of this shape are NP-hard in general — the number of feasible order-to-plant combinations grows exponentially with the number of orders and plants — which is why this project evaluates classical, heuristic, and quantum approaches rather than assuming one is sufficient, and why the comparison between them is the actual deliverable, not any single method in isolation.

Scale: 25,193 order-lines, 8 plants, 1,110 SKUs, of which 989 (89%) are carried at more than one plant — meaning genuine redirection across plants is possible for the large majority of the order book, not a theoretical exercise limited to a handful of edge cases.

Key trade-offs this report addresses directly, with measured numbers rather than assertions:

- **Solution quality vs. runtime** — an exact solver gives the provably best assignment; does it scale to the full order book, or only a subset? (Section 7 answers this directly: yes, in seconds.)
- **Exact solvers vs. heuristics** — how much value does a simple greedy rule capture versus a full exact solve, and is the added complexity of exact optimization worth it in practice? (Section 5.1 shows both, side by side.)
- **Current quantum hardware/simulators vs. classical** — does quantum offer any advantage on this problem class today, and if not, what is the honest, current role for it? (Section 4 addresses this without hedging.)

---

2. Data & Baselines

2.1 Data understanding

Each order-line carries the fields below. This structure is what makes genuine reassignment computable from real data, rather than requiring fabricated assumptions:

Field group	Key columns	Role
Order identity	`Group_Flag` (order ID), `MaterialNumber` (SKU), `Plant` (recorded default)	Identifies the decision unit
Demand	`OrderedQty_converted`, `Order_SKU_Revenue`	What's being asked for, and its value
Plant resources	`Available_inventory`, `Dock_Remaining`, `Throughput_Capacity`	What each plant can actually supply, per SKU
Cost	`Shipping_Cost`	Confirmed to be a single fixed rate per plant across the entire dataset — a real, observed truckload/route cost, not an individual per-order charge
Existing output	`Selected`, `Recommendation`, `Reason`	The dataset's own pre-existing assignment decision (audited in Section 5.2)

2.2 Baselines implemented

- **Baseline 1 (default assignment):** orders accepted in original arrival order against their own recorded plant's inventory, no redirection. This is the "business as usual" reference point every other method is measured against.
- **Baseline 2 (greedy reassignment):** orders processed by revenue (highest first); each tries its default plant, then the cheapest-shipping real alternate plant that carries the same SKU, before being rejected. This satisfies the challenge brief's requirement for "a simple greedy or sequential reassignment heuristic" as a genuine second baseline, not a restatement of Baseline 1.

A real per-plant shipping rate was confirmed empirically in the data before it was used in any model: `Shipping_Cost` takes exactly one distinct value per plant across all 25,193 rows. This means an order redirected to an alternate plant can use that plant's own already-observed rate directly — not an estimate, scaling factor, or assumption, which materially strengthens the reassignment results in Section 5.

---

3. Mathematical Formulation

Decision variable: $x_{ij} = 1$ if order *i* is assigned to plant *j*, for *j* ∈ the real set of plants that carry order *i*'s SKU anywhere in the dataset (not an arbitrary candidate list).

Objective (maximize): $Z = \Sigma_{ij}\ \text{Revenue}_i \cdot x_{ij}$

Revenue does not depend on which candidate plant fulfills an order, so shipping cost is not subtracted at full weight — it would incorrectly treat a shared truckload rate as an individual per-order cost. Instead, revenue is scaled by 10,000 and shipping cost subtracted as a tie-break, so cheaper-shipping plants are preferred whenever multiple candidates are equally revenue-optimal, without ever changing an accept/reject decision.

Constraints:

- One order → at most one plant: $\Sigma_j x_{\_} \leq 1$
- Inventory: Σ_i Demand_i · $x_{\_} \leq$ Available_inventory at plant *j* for that SKU

---

4. Quantum Implementation

The same problem, restricted to accept/reject at a single plant (x_i, not the full $x_{\_}$), is encoded as a QUBO via Qiskit Optimization's automatic converter (`QuadraticProgramToQubo`) — proper penalty terms, not hand-tuned coefficients — and solved with QAOA on a noiseless simulator.

QUBO conversion: a QUBO (Quadratic Unconstrained Binary Optimization) re-expresses a constrained problem with no explicit constraints — each constraint becomes a quadratic penalty term in the objective, so a quantum solver only has to minimize one expression. `QuadraticProgramToQubo` handles this automatically, including the integer slack variables needed to convert the inequality (≤) inventory constraint into an equality the QUBO form requires. This is the mechanism that drives the qubit count discussed in Section 7.

Instance: Plant 5385, SKU 12386067, capacity 8 units, 4 real orders (auto-selected: the largest tractable real group with a genuine capacity trade-off — total demand exceeds capacity, but more than one order individually fits, so there's an actual decision to make rather than a trivial accept-all or reject-all case).

Method	Objective	Feasible	Runtime
Exact (brute-force)	378	✔	0.0002s
Greedy heuristic	378	✔	0.0000s
QAOA (Qiskit simulator)	378	✔	~33s

QAOA is deterministic given a fixed starting point — a single run proves nothing about reliability. Running it 6 times from independently randomized starting points: **6 of 6 reached the exact optimum, 100% feasible.**

Why this doesn't demonstrate quantum advantage, and shouldn't be read as trying to: capacity-constrained assignment is exactly the structured, well-behaved combinatorial problem classical solvers already excel at. QAOA matching classical here is the expected outcome for a problem this size, not evidence of a broader speedup. The value of this result is a correctly-built, honestly-benchmarked pipeline — not a performance claim.

Hardware limitations: simulator only (no real QPU run yet); exponential simulation cost caps batch size (~8 qubits ≈ 30s, ~10-11 impractical, measured empirically); no optimality guarantee from QAOA itself, unlike the classical path.

Quantum scope note: QAOA currently solves only the accept/reject sub-case. The classical path was extended to genuine multi-plant reassignment (Section 5); doing the same for QAOA would multiply the qubit requirement by the average candidate-plant count (~4.9) — beyond the measured tractable budget. Stated as a scope limitation, not an inconsistency.

---

5. Results

5.1 Full-scale reassignment (all 25,193 orders)

Every order may go to any real plant carrying its SKU, not just its recorded default:

Method	Revenue	Shipping Cost	Runtime	Fill Rate	Orders Reassigned
Baseline (no reassignment)	$87,699,480	$61,349,438	1.5s	98.92%	0
Greedy reassignment	$88,088,551	$61,670,688	2.6s	99.54%	162
OR-Tools reassignment	$88,089,989	**$53,805,686**	2.7s	99.54%	18,899

Headline finding: the dataset's existing assignment achieves only ~17% fill rate and ~14% warehouse utilization. Solving the same data's constraints exactly (OR-Tools) reaches ~99% fill rate, in under 3 seconds — the shortfall is an assignment-logic gap, not an inventory shortage. Optimizing hard for shipping cost also has a real trade-off: it concentrates volume into cheap-shipping plants, dropping average utilization evenness from ~92% (single-plant OR-Tools) to a more concentrated distribution — cost efficiency and even network load are different goals.

5.2 Data-quality finding

An audit of the dataset's own pre-existing order-assignment column found it violates its own inventory capacity constraint in one case: Plant 5083 / SKU 12180656 has 319 units committed against 120 units of capacity — a 199-unit overage already present in the provided extract, not introduced by this analysis.

5.3 Business Interpretation

- **Revenue is not the bottleneck; shipping efficiency is.** The current process is already close to revenue-optimal (within ~0.4% of what full optimization achieves) — the real opportunity is *how* orders are fulfilled, not whether to accept more of them.
- **A small operational change captures most of the benefit.** Baseline 2 (simple greedy redirection) needs only 162 orders moved to lift fill rate from 98.92% to 99.54% — a realistic first step with limited disruption, versus the 18,899 orders the fully optimized solution redirects.
- **Cost optimization and network balance pull in different directions.** The $7.5M shipping-cost reduction comes with utilization concentrating into fewer, cheaper-to-ship-from plants. A planner who values network resilience over pure cost minimization may prefer a version with a cap on how much volume any single plant absorbs — the same tools used here support that variant directly.

---

6. Evaluation Methodology

All methods are compared on identical objective functions and feasibility checks — this is the standard this project holds itself to throughout, not just a stated intention:

- **Reassignment comparison** (Baseline / Greedy / OR-Tools): run on the full 25,193-order dataset, same revenue-maximizing objective, same per-(plant, SKU) inventory constraint, feasibility verified for every method (Baseline and OR-Tools are feasible by construction; Greedy's sequential capacity tracking is checked directly).

- **Quantum comparison** (Exact / Greedy / QAOA): run on one small batch all three methods can solve, same objective and constraint as above, restricted to the accept/reject sub-case. QAOA's feasibility is checked *after* solving against the real constraint — not assumed from the QUBO objective, since QUBO penalty terms are soft and a high-scoring QUBO solution can still violate the real constraint.
- **Reliability, not single-run results:** QAOA was run 6 times from independently randomized starting points rather than reported from one run, since it is deterministic given a fixed starting point and a single run cannot demonstrate reliable convergence.

Metrics reported throughout: revenue, shipping cost, fill rate, warehouse utilization, runtime, and — where reassignment applies — number of orders redirected from their original default plant. The two comparisons deliberately run at different scales (full dataset vs. one small batch) for a measured, stated reason (Section 7), not an unexplained inconsistency.

---

7. Scalability Analysis

Full multi-plant reassignment has ~124,000 binary variables (25,193 orders × ~4.9 average candidate plants) — OR-Tools solves this exactly in ~2-3 seconds. QAOA's qubit requirement is dominated by slack variables needed to encode the inventory inequality constraint; simulated runtime was measured empirically at ~5s (6 qubits), ~30s (8 qubits), and impractical past ~10-11 qubits in a CPU-only environment. This is the concrete, measured reason quantum stays scoped to a small batch, not an arbitrary choice.

---

8. Assumptions

- Candidate plants for reassignment are restricted to plants that genuinely carry the order's SKU elsewhere in the dataset (confirmed: 89% of SKUs qualify) — not an unconstrained "any of the 8 plants" assumption.
- Shipping cost for a reassigned order uses that plant's own real, already-observed rate.
- `Risk_Score` is empty in this extract and treated as unavailable, not imputed.

---

9. Limitations

- Quantum implementation covers accept/reject only, not full reassignment (Section 4).
- QAOA runs on a noiseless simulator; no real hardware run yet.
- QUBO penalty terms are soft; feasibility is checked post-hoc, not guaranteed.
- No optimality certificate from QAOA — only empirical comparison against exact enumeration on small instances.
- Multi-plant *split* fulfillment (one order sourced from two plants at once) is out of scope; each order still goes to exactly one plant.

---

10. Future Work

1. Real IBM Quantum hardware run (migration path scoped; blocked only on account credentials).

2. Extend QAOA to the reassignment model on a reduced candidate-plant subset.
3. Investigate constraint-preserving QAOA mixers to remove slack-variable qubit overhead.
4. Automate cache regeneration in CI as underlying data changes.
5. Resolve the found constraint violation (Section 5.2) with the data source.
6. Model multi-plant split-fulfillment for partially-filled orders.

Full technical documentation, including architecture diagrams, repository structure, and complete derivations: `Nestle_DOM_Documentation.md`. *Business-language summary:* `Planner_View.md`.